

Tema 6: Árboles de Decisión

Aprendizaje Supervisado — Regresión y Clasificación

Técnicas de Aprendizaje Automático

Maestría en Inteligencia Artificial — UNIR

Complemento: CS229 Stanford (Andrew Ng) — Lectures 1–4

Mayo 2026

Índice

1. Introducción e intuición de los árboles de decisión
2. Anatomía del árbol: raíz, nodos internos y hojas
3. Interpretación geométrica
4. Criterios de división: Entropía, InfoGain e Índice de Gini
5. Árboles de regresión
6. Algoritmos clásicos: ID3, C4.5, C5.0, CART
7. Overfitting y poda (pre poda y pospoda)
8. Árboles vs. modelos lineales
9. Comandos Python clave (scikit-learn)
10. Ejercicio resuelto: Buy Computer (UNIR)
11. Preguntas tipo examen con respuestas
12. Conexión con CS229 Stanford

1. Introducción e intuición

Un árbol de decisión es un modelo de aprendizaje automático supervisado que predice una salida (clase o valor numérico) navegando una secuencia jerárquica de preguntas sobre los atributos del dato de entrada.

Definición formal

Un árbol de decisión es un clasificador (o regresor) expresado como una *partición recursiva del espacio de instancias*. Cada nodo interno divide el espacio de features según una condición sobre un atributo; cada hoja asigna una predicción final.

Analogía cotidiana — ¿Llevamos paraguas hoy?

1. ¿Está nublado? → Si No: no llevo paraguas (hoja).
2. ¿Probabilidad de lluvia > 60%? → Si Sí: llevo paraguas (hoja).
3. ¿Voy a estar afuera más de 2 horas? → Si Sí: llevo paraguas.

Cada pregunta evalúa un **atributo (feature)**; la conclusión final corresponde a la **variable objetivo (target)**.

Los árboles de decisión sirven tanto para **clasificación** (target categórico: Sí/No, clase A/B/C) como para **regresión** (target numérico continuo: precio, salario, temperatura). Son uno de los modelos más interpretables del Machine Learning.

2. Anatomía del árbol

Tres tipos de nodos

- **Nodo raíz (root node):**

único nodo sin aristas entrantes. Es el punto de entrada; contiene la pregunta más informativa del árbol.

- **Nodos internos (nodos de decisión / de prueba):**

tienen una arista entrante y dos o más salientes. Cada uno contiene una condición evaluable sobre un atributo (ej: $\text{¿edad} \leq 30?$). Las ramas que salen representan las posibles respuestas.

- **Nodos hoja (nodos terminales):**

tienen una arista entrante y ninguna saliente. Contienen la predicción final:

- En *clasificación*: clase mayoritaria de las observaciones de entrenamiento que cayeron en esa hoja (opcionalmente un vector de probabilidades).
- En *regresión*: media del target de las observaciones de entrenamiento que cayeron en esa hoja.

Distinción nodo / rama

Los **nodos** contienen *preguntas* (ej: ¿tarifa > 50?). Las **ramas** contienen *respuestas* (Sí / No). Las **hojas** contienen *predicciones*. Confundir estos tres elementos es el error más frecuente en exámenes.

Construcción del árbol (algoritmo greedy)

El árbol se construye de manera **recursiva** y **top-down**:

1. Evaluar todos los atributos candidatos y elegir el que produce la mayor reducción de impureza (mayor InfoGain o mayor Δ Gini).
2. Dividir el nodo en subnodos según los valores del atributo elegido.
3. Repetir para cada subnodo hasta alcanzar un criterio de parada.

Criterios de parada

- Todos (o casi todos) los ejemplos del nodo son de la misma clase → nodo puro, se convierte en hoja.
- No existen atributos disponibles para dividir → se convierte en hoja con la clase mayoritaria.
- El árbol alcanzó una complejidad predefinida (profundidad máxima, mínimo de muestras, etc.).

3. Interpretación geométrica

Cada condición de un nodo interno equivale a un **corte ortogonal a uno de los ejes** del espacio de features. El resultado es que el árbol particiona el espacio en **hiperrectángulos** (rectangulares en 2D, cúbicos en 3D, etc.).

Ejemplo en 2D (Titanic)

Con features *edad* (eje X) y *tarifa* (eje Y):

- Pregunta ¿ $edad \leq 30$? → corte vertical en $x = 30$.
- Pregunta ¿ $tarifa \leq 50$? → corte horizontal en $y = 50$.

El resultado son cuatro regiones rectangulares, cada una asignada a una clase predicha.

Cada hoja = una región rectangular.

Implicación clave: frontera de decisión no lineal

Los árboles pueden capturar **fronteras de decisión no lineales** formadas por reglas con conjunciones lógicas (AND/OR), porque cada AND corresponde a una subdivisión adicional. Los modelos lineales, en cambio, solo pueden trazar una única frontera lineal (recta, plano o hiperplano).

Limitación: no extrapolación

Los árboles solo predicen dentro del rango de valores vistos en entrenamiento. Una hoja predice la media (o clase mayoritaria) de su región — si un nuevo dato cae fuera del rango conocido, el árbol asigna la predicción del borde más cercano, sin extrapolar.

4. Criterios de división

4.1 Entropía de Shannon

Entropía

Mide el *desorden* o *impureza* de un nodo. Cuanto más mezcladas estén las clases, mayor es la entropía.

$$H(S) = - \sum_{i=1}^c p_i \cdot \log_2(p_i)$$

Donde:

- S = conjunto de observaciones del nodo.
- c = número de clases del problema.
- p_i = frecuencia relativa (proporción) de la clase i en S .
- $\log_2$ = logaritmo en base 2 (unidad: bits). Por convención:

$$0 \cdot \log_2(0) = 0 \quad .$$

Casos extremos (clasificación binaria)

Nodo puro — todas las observaciones son de la misma clase

($p_1 = 1, p_2 = 0$):

$$H = -[1 \cdot \log_2(1) + 0 \cdot \log_2(0)] = -[0 + 0] = 0$$

Nodo máximamente impuro — mitad de cada clase ($p_1 = p_2 = 0.5$):

$$H = -[0.5 \cdot \log_2(0.5) + 0.5 \cdot \log_2(0.5)] = -[0.5 \cdot (-1) + 0.5 \cdot (-1)] = 1$$

Rango en binario: $H \in [0, 1]$. En general, con c clases:

$$H \in [0, \log_2(c)] \quad .$$

4.2 Ganancia de Información (InfoGain)

Ganancia de Información

Mide cuánto se reduce la entropía al dividir el nodo por el atributo F .

$$\text{InfoGain}(F) = H(S) - \sum_{v \in \text{valores}(F)} \frac{|S_v|}{|S|} \cdot H(S_v)$$

Donde:

- $H(S)$ = entropía del nodo *antes* de dividir.
- v = cada valor posible del atributo F .
- S_v = subconjunto de observaciones con $F = v$.
- $|S_v|/|S|$ = peso del subnodo (proporción del total).
- $H(S_v)$ = entropía del subnodo v .

El algoritmo elige el atributo F^* que **maximiza** InfoGain:

$$F^* = \arg \max_F \text{InfoGain}(F)$$

Regla de oro

Mayor InfoGain → mayor reducción de entropía → mejor separación de clases → ese atributo va en el nodo. La entropía (impureza) se *minimiza*; la ganancia se *maximiza*. Son métricas en direcciones opuestas.

4.3 Índice de Gini

Índice de Gini

Mide la *impureza* de un nodo como la probabilidad de clasificar incorrectamente una observación elegida al azar si se le asigna una clase aleatoriamente según la distribución del nodo.

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

Donde p_i es la frecuencia relativa de la clase i en S .

Casos extremos (clasificación binaria)

Nodo puro ($p_1 = 1, p_2 = 0$):

$$\text{Gini} = 1 - [1^2 + 0^2] = 0$$

Nodo máximamente impuro ($p_1 = p_2 = 0.5$):

$$\text{Gini} = 1 - [0.5^2 + 0.5^2] = 1 - 0.5 = 0.5$$

Rango en binario: $\text{Gini} \in [0, 0.5]$.

Reducción de Gini (ΔGini)

Análogo al InfoGain pero usando el índice de Gini como medida de impureza:

$$\Delta\text{Gini}(F) = \text{Gini}(S) - \sum_v \frac{|S_v|}{|S|} \cdot \text{Gini}(S_v)$$

El algoritmo elige el atributo que **maximiza** ΔGini .

4.4 Comparación Entropía vs. Gini

Aspecto	Entropía + InfoGain	Índice de Gini + ΔGini
Fórmula	$-\sum p_i \log_2(p_i)$	$1 - \sum p_i^2$
Rango (binario)	$[0, 1]$	$[0, 0.5]$

Mínimo (puro)	0	0
Algoritmos	ID3, C4.5, C5.0	CART (default sklearn)
Costo computacional	Mayor (logaritmos)	Menor (solo cuadrados)
Uso típico	Didáctica, interpretación	Producción, Random Forest

No existe un criterio universalmente superior. La diferencia en calidad del modelo suele ser pequeña. Se recomienda probar ambos y evaluar en conjunto de validación (Aning y Przybyła-Kasperek, 2022).

5. Árboles de Regresión

Cuando el target es numérico continuo, la estructura del árbol es idéntica pero cambian dos elementos:

Aspecto	Clasificación	Regresión
Predicción de la hoja	Clase mayoritaria	Media del target ($\bar{y}_S$)
Métrica de impureza	Entropía / Gini	Varianza (o MSE)
Criterio de división	Maximizar InfoGain / Δ Gini	Maximizar reducción de varianza
Algoritmo típico	ID3, C4.5, C5.0, CART	Solo CART

Varianza del nodo (criterio para regresión)

$$\text{Var}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$$

Donde y_i es el valor real del target para la observación i y $\bar{y}_S$ es la

media del target en el nodo S . Esta es exactamente la fórmula del MSE (Mean

Squared Error) del Tema 4, usando la media del nodo como predicción. **Reducción de varianza:**

$$\Delta \text{Var}(F) = \text{Var}(S) - \sum_v \frac{|S_v|}{|S|} \cdot \text{Var}(S_v)$$

¿Por qué no usar entropía en regresión?

La entropía requiere clases discretas para calcular proporciones p_i . Con un target

continuo, cada observación puede tener un valor único — habría tantas "clases" como instancias, y la entropía daría valores altos constantes sin discriminar buenas de malas divisiones. La varianza, en cambio, mide *distancias cuadráticas* entre valores, capturando adecuadamente la cercanía numérica.

6. Algoritmos Clásicos

Algoritmo	Año	Autor	Clasif.	Regr.	Var. cat.	Var. num.	Criterio	Tipo árbol
ID3	1986	Quinlan	✓	✗	✓	✗	InfoGain	n-ario
C4.5	1993	Quinlan	✓	✗	✓	✓	Gain ratio	mixto

C5.0	post-1993	Quinlan	✓	✗	✓	✓	InfoGain mejorado	mixto
CART	1984	Breiman et al.	✓	✓	✓	✓	Gini / Varianza	binario

Características clave de cada algoritmo

- **ID3:**

primer algoritmo histórico, solo variables categóricas, solo clasificación. Sin soporte para continuos ni valores faltantes.

- **C4.5:**

extiende ID3 con soporte para variables numéricas (definiendo puntos de corte dinámicamente). Convierte el árbol en reglas if-then y aplica poda eliminando condiciones que no mejoran la precisión.

- **C5.0:**

versión comercial (licencia propietaria) de C4.5. Usa menos memoria, genera conjuntos de reglas más pequeños y es más preciso.

- **CART:**

único que soporta regresión. Siempre genera árboles *estrictamente binarios* (cada nodo tiene exactamente 2 hijos). Es el algoritmo detrás de `DecisionTreeClassifier` y `DecisionTreeRegressor` de scikit-learn.

Trampas frecuentes en exámenes

- "¿Qué algoritmo sirve para regresión?" → Solo **CART**.
- "¿Cuál es el criterio default de scikit-learn?" → **Gini** (CART).
- "¿Cuál convierte el árbol en reglas if-then?" → **C4.5**.
- "¿Cuál fue el primero?" → **ID3** (**1986**) según el material UNIR (aunque CART es de 1984).
- "¿Cuál genera árboles binarios siempre?" → **CART**.

7. Overfitting y Poda

7.1 Por qué los árboles sobreajustan

El algoritmo greedy puede seguir dividiendo hasta que cada hoja contenga una sola observación de entrenamiento. En ese estado, el árbol tiene **accuracy en train = 100%**, pero ha memorizado el ruido del dataset — su capacidad de generalización a datos nuevos es pobre.

Manifestación del bias-variance tradeoff en árboles

- **Árbol muy profundo (sin podar):**

baja impureza en train, alta varianza → overfitting. Frontera de decisión caótica (miles de hiperrectángulos minúsculos).

- **Árbol muy poco profundo:**

alta impureza residual, alto sesgo → underfitting. Frontera de decisión demasiado simple.

- **Árbol óptimo:**

profundidad que minimiza error en validación. Se encuentra con poda o búsqueda de hiperparámetros.

$$\text{Error total} = \text{Sesgo}^2 + \text{Varianza} + \text{Error irreducible}$$

Adicionalmente, los árboles son **inestables**: pequeñas variaciones en los datos pueden cambiar completamente la pregunta del nodo raíz y, por cascada, toda la estructura del árbol (alta varianza estructural). Este problema motiva el uso de *ensembles* como Random Forest.

7.2 Prepoda (Pre-Pruning / Early Stopping)

Prepoda

Detiene el crecimiento del árbol durante su construcción aplicando criterios de parada tempranos. Parámetros típicos:

- `max_depth` : profundidad máxima del árbol.
- `min_samples_split` : mínimo de muestras requeridas para dividir un nodo.
- `min_samples_leaf` : mínimo de muestras requeridas en una hoja.
- `min_impurity_decrease` : ganancia mínima requerida para realizar una división.

Ventaja: computacionalmente eficiente (no construye ramas innecesarias).

Desventaja: visión miope (greedy local) — puede descartar divisiones que a corto plazo parecen malas pero que a largo plazo serían valiosas.

7.3 Pospoda (Post-Pruning)

Pospoda

Construye el árbol completo y luego elimina ramas que no mejoran el rendimiento en un conjunto de validación.

1. Construir árbol completo hasta pureza total.
2. Evaluar cada subárbol candidato a eliminar sobre datos de validación.
3. Si quitar el subárbol no empeora (o mejora) el rendimiento: eliminarlo y reemplazarlo por una hoja.
4. Repetir hasta que no haya mejoras posibles.

Ventaja: visión global — el árbol completo exploró todas las divisiones posibles antes de podar.

Desventaja: computacionalmente más costoso.

Aspecto	Prepoda	Pospoda
¿Cuándo se aplica?	Durante la construcción	Después de construir el árbol completo
Visión	Local / miope (greedy)	Global
Costo computacional	Bajo	Alto

Riesgo principal	Underfitting (cortar de más)	Construir árbol gigante innecesariamente
Calidad del modelo	Buena	Generalmente superior
Datos requeridos	Solo train	Train + validación

Mantra de la poda (para el examen)

La poda **sacrifica ajuste en train** (acepta un poco más de sesgo) para **mejorar generalización en test** (reducir varianza). Cualquier opción que diga que la poda "mejora el ajuste a los datos de entrenamiento" es **falsa**.

7.4 Selección del árbol óptimo

- **Validación cruzada (k-fold CV):** estrategia más robusta. Evalúa el rendimiento promedio sobre k particiones del dataset.
- **Curva de validación:** graficar error vs. complejidad del árbol (profundidad o número de hojas). El punto óptimo es donde la curva de validación alcanza su mínimo antes de volver a subir.
- **Búsqueda de hiperparámetros:** `GridSearchCV` con `max_depth`, `min_samples_split`, etc.

8. Árboles vs. Modelos Lineales

Aspecto	Modelos Lineales (Tema 4)	Árboles de Decisión
Frontera de decisión	Lineal (recta / hiperplano)	No lineal (hiperrectángulos ortogonales)
Relaciones que captura	Solo lineales	No lineales, interacciones AND/OR
Escalado de features	Requerido (StandardScaler)	No requerido
Variables categóricas	Requiere One-Hot Encoding	Soporta nativamente (CART, C4.5)
Supuestos estadísticos	Linealidad, normalidad, homocedasticidad, independencia	Ninguno (distribution-free)

Riesgo de overfitting	Bajo (con regularización)	Alto sin poda
Estabilidad	Alta	Baja (sensible a variaciones en datos)
Extrapolación	Sí	No
Interpretabilidad	Buena (coeficientes)	Excelente (reglas if-then visualizables)

Regla práctica del material UNIR

Si las relaciones entre variables se pueden aproximar por un modelo lineal → la **regresión lineal supera al árbol**.

Si la naturaleza del problema es no lineal y con relaciones complejas → los **árboles superan a los modelos lineales**.

En la práctica de producción, casi nunca se usan árboles individuales — se usan *ensembles* (Random Forest, XGBoost, LightGBM) que promedian muchos árboles para estabilizar las predicciones.

9. Comandos Python clave (scikit-learn)

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.tree import export_text, plot_tree
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
import matplotlib.pyplot as plt

# --- Clasificación ---
clf = DecisionTreeClassifier(
    criterion='gini',          # 'gini' (default CART) o 'entropy' (InfoGain)
    max_depth=5,              # prepoda: profundidad máxima
    min_samples_split=10,     # prepoda: mínimo muestras para dividir
    min_samples_leaf=5,       # prepoda: mínimo muestras por hoja
    random_state=42
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_proba = clf.predict_proba(X_test) # probabilidades por clase

# --- Regresión ---
reg = DecisionTreeRegressor(
    criterion='squared_error', # reducción de varianza (MSE)
```



```

        max_depth=5,
        random_state=42
    )
    reg.fit(X_train, y_train)

# --- Visualización del árbol ---
print(export_text(clf, feature_names=feature_names))
plot_tree(clf, feature_names=feature_names, class_names=class_names, filled=True)
plt.show()

# --- Búsqueda de profundidad óptima (pospoda guiada por CV) ---
param_grid = {'max_depth': [3, 5, 8, 10, 15, 20]}
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)
print(f"Mejor profundidad: {grid.best_params_}")

# --- Importancia de features ---
importances = clf.feature_importances_ # reducción total de Gini/InfoGain por feature

```

10. Ejercicio Resuelto: Buy Computer (UNIR)

Dataset de 14 instancias con atributos: *age*, *income*, *student*, *credit_rating*. Variable objetivo: *buy_computer* (9 Yes, 5 No).

Paso 1: Entropía del nodo raíz

$$H(S) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) \approx 0.940\text{bits}$$

Paso 2: InfoGain de cada atributo

Atributo: age

Valores: youth (5 instancias: 2 Yes, 3 No), middle_age (4: 4 Yes, 0 No), senior (5: 3 Yes, 2 No).

$$H(\text{youth}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} \approx 0.971$$

$$H(\text{middle_age}) = 0 \quad (\text{nodopuro : 4Yes, 0No})$$

$$H(\text{senior}) \approx 0.971 \quad (\text{simétrico a youth})$$

Promedio ponderado:

$$H(S | \text{age}) = \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) = 0.694$$

$$\text{InfoGain}(\text{age}) = 0.940 - 0.694 = 0.246$$

Paso 3: Comparación de ganancias

Atributo	InfoGain	Resultado
age	0.246	Ganador → nodo raíz
student	0.152	—
credit_rating	0.048	—
income	0.029	—

Paso 4: División por age y árbol resultante

- **age = middle_age**: entropía = 0 (todos Yes) → hoja: `class = YES`.
- **age = youth**: 5 instancias mezcladas → se repite el proceso. Ganador: *student* (InfoGain = 0.97). Resultado: `student=yes → YES`, `student=no → NO`.
- **age = senior**: 5 instancias mezcladas → se repite el proceso. Ganador: *credit_rating* (InfoGain = 0.97). Resultado: `credit_rating=fair → YES`, `credit_rating=excellent → NO`.

Predicción del caso de prueba

Instancia: `age=youth, income=medium, student=yes, credit_rating=fair`

1. Nodo raíz: `age = youth` → rama youth.
2. Nodo interno: `student = yes` → rama yes.
3. Hoja: `class = YES` → el modelo predice que **compra la computadora**.

11. Preguntas tipo examen con respuestas

P1. ¿Cuál es la diferencia entre un nodo interno y una hoja en un árbol de decisión?

Un **nodo interno** contiene una condición evaluable sobre un atributo (ej: `¿edad ≤ 30?`) y tiene aristas salientes hacia subnodos. Una **hoja** no tiene aristas salientes y contiene la predicción final: clase mayoritaria (clasificación) o media del target (regresión).

P2. ¿Por qué el índice de Gini = 0.5 en clasificación binaria con distribución 50-50?

Con $p_1 = p_2 = 0.5$:

$$\text{Gini} = 1 - [0.5^2 + 0.5^2] = 1 - 0.5 = 0.5 \quad . \text{ Este es el máximo}$$

posible en binario. Cualquier otro reparto (ej: 70-30) produce $\text{Gini} < 0.5$

porque $0.7^2 + 0.3^2 = 0.58 > 0.5$, lo que da $\text{Gini} = 0.42$.

P3. ¿Por qué los árboles de decisión son propensos al overfitting?

Si se dejan crecer sin restricciones, el algoritmo greedy puede seguir dividiendo hasta que cada hoja contenga una sola observación (pureza perfecta en train). El árbol memoriza el dataset de entrenamiento incluyendo el ruido, produciendo una frontera de decisión caótica que no generaliza a datos nuevos.

P4. ¿Cuál es la principal diferencia entre prepoda y pospoda?

La **prepoda** aplica criterios de parada *durante* la construcción para limitar el crecimiento (miope, eficiente). La **pospoda** construye el árbol completo y luego elimina ramas que no contribuyen al rendimiento en validación (visión global, mayor calidad, más costosa). La pospoda generalmente produce mejores modelos.

P5. ¿Por qué no se usa entropía como criterio de división en árboles de regresión?

La entropía requiere categorías discretas para calcular proporciones p_i . Con un target continuo, cada valor es prácticamente único — se necesitarían tantas "clases" como instancias, y la entropía no discriminaría entre buenas y malas divisiones. La **varianza** (o MSE) es la métrica correcta porque mide distancias cuadráticas entre valores numéricos, capturando la cercanía numérica que importa en regresión.

P6. ¿En qué situación se prefiere un árbol sobre regresión logística?

Cuando: (1) el problema tiene **fronteras de decisión no lineales** (reglas con AND/OR entre features), (2) las features son **mixtas** (categóricas y numéricas), (3) se busca **interpretabilidad visual** (reglas if-then), (4) no se quiere realizar preprocesamiento extensivo (escalado, codificación). La regresión logística es preferible cuando la frontera es aproximadamente lineal y se busca interpretabilidad estadística (coeficientes con significancia).

P7. ¿Cuál de los algoritmos clásicos es el único que soporta regresión?

CART (Classification and Regression Trees, Breiman et al., 1984). Es también el único que genera árboles estrictamente binarios y el que implementa scikit-learn (`DecisionTreeClassifier` y `DecisionTreeRegressor`).

12. Conexión con CS229 Stanford

Vínculos con el curso de Andrew Ng

- **Descenso por gradiente (Lectures 1–2):** CART minimiza la impureza de forma greedy en cada nodo — es una optimización local sin garantía de óptimo global, análogo al riesgo de mínimos locales en gradiente descendente.
- **Bias-variance tradeoff (Lecture 8):** profundidad del árbol controla directamente el tradeoff: árboles profundos = alta varianza (overfitting); superficiales = alto sesgo (underfitting). La poda busca el punto óptimo, igual que la regularización en modelos lineales.
- **Ensembles (Lectures 13–14):** Random Forest promedia muchos árboles con subconjuntos aleatorios de features para reducir varianza sin aumentar sesgo. Gradient Boosting encadena árboles que corrigen los errores del anterior. Ambos tienen como bloque básico el árbol de decisión estudiado en este tema.
- **Regularización:** la poda es la forma de regularización de los árboles, análoga al término λ en Ridge/Lasso del Tema 4. Ambas estrategias sacrifican ajuste en train para mejorar generalización en test.